Clothing Mill Production & Wage Management System — Project Plan

0. Confirmed Requirements

These were the open questions from the first draft of this plan — all now confirmed with the client, so the rest of this document is written as a finalized plan rather than a set of assumptions.

1. **Wage rate structure** — One flat rate per yard for everyone: **2.69 BDT/yard**. Still stored as an editable, date-stamped setting rather than hardcoded (Section 3.4), so the owner can update it later without affecting past payslips.
2. **Multiple workers per machine** — Confirmed: more than one worker can be logged against the same machine on the same day (e.g., shift handover).
3. **Users and roles** — Exactly two roles: **Admin** and **Manager**. No accountant/view-only role needed.
4. **Absences and holidays** — Not tracked. The system only logs entries for workers who actually worked; no leave/absence module.
5. **Advances and deductions** — Not tracked. Wage = yard × rate, with no adjustment fields for advances or deductions.
6. **Thread-change data** — Purely informational. It's recorded as part of the daily entry (matching the paper register) but doesn't drive any maintenance, quality, or bonus/penalty logic. No dedicated analytics report needed for it — see Section 3.5.
7. **Historical data import** — Not decided yet. Treated as an open item for later (Section 8) — doesn't block initial development, since the system can start fresh from go-live and historical paper records can be backfilled afterward if needed.
8. **Currency/rounding** — Wages in BDT. Rounding convention and any minimum-wage-floor check are left as a small open detail — confirm rounding to nearest taka (vs. paisa) before Phase 3.

---

1. Project Overview

The client currently runs a manual, paper-based register. The website's job is to **digitize the daily entry process** (date → machine → worker(s) → yards produced → thread-change points) and **automatically derive** everything that today is calculated by hand: per-machine output, per-worker output, wages, and monthly summaries — with a printable invoice/payslip at the end of the pipeline.

The core insight from the example given (`25+29+12...`) is that a single worker's daily total is actually a **sum of segments**, where each segment ends when a new thread (shuta) is loaded. That segment breakdown is valuable data, not just a formality — it doubles as a record of how often thread changes happen per machine/worker, which can later feed into maintenance or quality metrics. The data model below preserves both the segments and their total.

---

2. Core Entities (Data Model)

Entity	Key Fields	Notes
Machine	machine number, type/model (optional), status (active/under repair)	One row per physical machine in the mill
Worker	name, worker ID, joining date, status (active/inactive)	No wage grade needed — rate is flat for everyone
Wage Rate	rate per yard (currently 2.69 BDT), effective-from date	A single flat rate applied to everyone, but still stored as a date-stamped setting so updates don't affect past payslips
Production Entry	date, machine, worker, list of segment yards (e.g. [25, 29, 12]), computed total yard for that entry	This is the digital replacement for one line in the paper register
Thread Change Log	derived from each segment boundary in a Production Entry	Each segment = yards produced before that thread change. No separate manual entry needed — it's a byproduct of how production is recorded
Invoice / Wage Slip	worker, period (month), list of daily entries in that period, total yard, total wage, generated date	Generated on demand, not stored as duplicate data — calculated fresh from Production Entries each time, then optionally saved as a PDF snapshot

This structure means **one daily data-entry action captures requirements 1, 2, and 3 simultaneously** — the manager doesn't fill out three separate forms.

---

3. Feature List

3.1 Daily Production Entry

The main day-to-day screen. For a selected date, the manager picks a machine, then for each worker on that machine enters the segment yards (e.g., types 25, then 29, then 12 — each one representing output before a thread change). The system auto-sums these into the day's total for that worker on that machine. Entry should be fast — large touch-friendly number inputs, ability to add a new segment with one tap, and a "save & add next worker" flow so a manager can move through an entire machine's crew quickly.

3.2 Machine Management

A simple list/CRUD screen for adding machines, marking one as under repair (so it's excluded from "expected to be running" counts but historical data stays intact), and viewing a machine's own production history.

3.3 Worker Management

A list/CRUD screen for adding workers, recording joining date, and marking workers inactive when they leave (so their historical wage records remain but they stop appearing in new-entry dropdowns).

3.4 Wage Rate Configuration

An admin-only settings page to set/update the flat rate per yard (currently 2.69 BDT). Rate changes should be **date-stamped**, not overwritten — i.e., if the rate changes mid-month, entries before the change date use the old rate and entries after use the new one, so historical payslips stay accurate even after the rate is updated.

3.5 Thread Change Tracking (built into entry, requirement 3)

No separate screen needed — it's captured automatically from the segments entered in 3.1, and displayed alongside each entry for record-keeping. Since this data is purely informational (confirmed with the client — it doesn't drive maintenance, quality, or bonus/penalty logic), no dedicated analytics report is needed for it; it just needs to be visible wherever a production entry is viewed or printed, matching what the paper register already shows.

3.6 Wage Calculation Engine

Runs automatically, not as a manual step:

- **Per entry:** `total yard = sum of segments`
- **Per worker per day:** sum of that worker's entries that day (in case they worked more than one machine)
- **Per worker per month:** sum of all their daily totals in the selected month
- **Wage for the period:** `total yard × applicable rate per yard` (using the rate that was active on each entry's date, per 3.4)

3.7 Daily Reports

For any selected date: total yards produced mill-wide, broken down by machine, with worker-level detail available by expanding a machine row.

3.8 Monthly Reports

For any selected month (auto-detecting 30 vs 31 days, and February's 28/29):

- Total mill output for the month
- Output per machine for the month
- Output and wage per worker for the month
- A simple bar/line chart of daily output trend across the month (nice-to-have, not core)

3.9 Invoice / Wage Slip Generation & Print

For a selected worker and month, generate a clean payslip: mill name/logo header, worker name and ID, the period, a day-by-day table (date, machine, yard produced), total yard, rate applied, total wage, and a signature line. Includes a "Print" button (browser print, styled for clean paper output) and a "Download PDF" option. The same layout pattern can produce a **mill-wide monthly summary invoice** (total yard, total wage payout, per-machine breakdown) for the owner's own records.

3.10 Dashboard

A landing page after login showing today's entry status (which machines have been logged, which haven't), this month's running total yard and wage liability so far, and quick links to enter today's data or jump to last month's report.

3.11 Authentication & Roles

Login system with exactly two roles, as confirmed with the client: **Admin** (full access, including wage rate settings, user management, and all reports) and **Manager** (can enter daily production data and view reports, but can't change wage rates or delete historical records).

---

4. Example Calculation Walkthrough

Using the client's own example: on 10/25, machine 47 had a worker who produced 25, then 29, then 12 yards before each thread change.

- Worker's total for that entry: `25 + 29 + 12 = 66 yards`

- That 66 contributes to: machine 47's daily total, the worker's daily total, and (at month end) both machine 47's monthly total and the worker's monthly total
- At the confirmed rate of **2.69 BDT/yard**, this entry's wage contribution is `66 × 2.69 = 177.54 BDT`
- The thread-change segments (25, 29, 12) are stored and shown alongside the entry for record-keeping, but — per the client — don't feed into any separate calculation or report.

---

5. Suggested Page/Screen List

1. Login
2. Dashboard (today's status + quick actions)
3. Daily Entry (the core data-entry screen)
4. Machines (list + add/edit + per-machine history)
5. Workers (list + add/edit + per-worker history)
6. Wage Rate Settings (admin only)
7. Daily Report
8. Monthly Report
9. Worker Payslip / Invoice (generate + print)
10. Mill Summary Invoice (generate + print)
11. User Management (admin only — managing the small number of Admin/Manager accounts)

---

6. Technical Recommendations

- **Stack:** A standard web stack works well here — React (or Next.js) for the frontend, a Node/Express or Next.js API backend, and PostgreSQL or MySQL for the database. Given you're a CSE student likely comfortable with a JS-based stack, Next.js end-to-end (frontend + API routes) with PostgreSQL keeps the project in one codebase and is easy to deploy.
- **Printing:** Use a dedicated print stylesheet (`@media print` CSS) for the invoice/payslip pages rather than building a separate PDF pipeline first — it's simpler and the browser's "Print to PDF" covers the "download" need too. If a polished PDF download button is required specifically, a library like Puppeteer (server-side) or jsPDF (client-side) can generate one from the same template later.
- **Data entry resilience:** Mills can have unreliable power/internet. Consider designing the entry form to hold unsaved input in local state and warn before navigating away, and keep the entry flow lightweight enough to redo quickly if a save fails.

- **Audit trail:** Since this replaces a paper register that's presumably treated as a source of truth, log who entered/edited each Production Entry and when — useful for resolving wage disputes later.
- **Validation:** Put sane bounds on yard inputs (e.g., flag entries that look unrealistically high for a single segment) to catch typos before they affect a worker's pay.

---

7. Suggested Build Order (Phases)

1. **Phase 1 — Core data + entry:** Machines, Workers, Daily Production Entry, basic auth.
2. **Phase 2 — Calculations + reports:** Wage rate settings, daily/monthly aggregation, daily & monthly reports.
3. **Phase 3 — Invoicing:** Worker payslip and mill summary invoice, with print/PDF.
4. **Phase 4 — Polish:** Dashboard, roles/permissions refinement, charts.

This order means there's a usable product (replacing the paper register) after Phase 1–2, even before invoicing is built — useful if you want to show the client early progress.

---

8. Remaining Open Item

Everything needed to start building is confirmed except one: whether the old paper register needs to be digitized/imported, or whether the system starts fresh from go-live date. This doesn't block Phase 1–2 development — it only matters before Phase 3 (invoicing), since the client may want historical months reflected in reports. Worth a quick follow-up question once the client has had a chance to think it over.